CASE STUDY: IMPLEMENTATION OF STACK AND QUEUE OPERATIONS USING LINKED LISTS

1. Introduction

Stack and Queue are fundamental linear data structures used in computer science to store and manage data. They can be implemented using arrays or linked lists.

A **linked list** is a dynamic data structure consisting of nodes. Each node contains data and a pointer (reference) to another node.

A **Stack** follows the LIFO (Last In, First Out) principle, meaning the last element inserted is the first element removed.

A **Queue** follows the FIFO (First In, First Out) principle, meaning the first element inserted is the first element removed.

This case study demonstrates how stack and queue operations can be implemented efficiently using linked lists.

2. Problem Statement

Static data structures may require a fixed capacity or resizing when the number of elements changes. The objective is to implement stack and queue operations using linked lists so that memory can be allocated dynamically as needed.

The system should support insertion, deletion, and display operations while maintaining the correct order of elements.

3. Objectives

- To understand linked lists and their node structure.
- To implement stack operations using a linked list.
- To implement queue operations using a linked list.
- To perform insertion, deletion, and display operations.
- To analyze the time and space complexity of each operation.
- To understand the practical applications of stacks and queues.

4. Proposed System

The proposed system uses a singly linked list to implement both data structures.

Each node contains:

- **Data:** The value stored in the node.
- **Next pointer:** A reference to the next node.

Stack Representation

A stack maintains a pointer called `top`, which points to the most recently inserted element.

```
TOP
 |
 v
[30 | *] -> [20 | *] -> [10 | NULL]
```

Insertion and deletion take place at the top of the stack.

Queue Representation

A queue maintains two pointers: `front` and `rear`.

```
FRONT                          REAR
  |                              |
  v                              v
[10 | *] -> [20 | *] -> [30 | NULL]
```

Insertion takes place at the rear, while deletion takes place at the front.

5. Stack Operations Using a Linked List

The main stack operations are:

1. **Push:** Insert an element at the top.
2. **Pop:** Remove the top element.
3. **Peek:** View the top element without removing it.
4. **Display:** Print all elements from top to bottom.
5. **is_empty:** Check whether the stack contains any elements.

Algorithm for Push

1. Create a new node containing the given value.
2. Set the new node's next pointer to the current top.
3. Update top to point to the new node.

Algorithm for Pop

1. Check whether the stack is empty.
2. If empty, report stack underflow.
3. Otherwise, store the top element.
4. Move top to the next node.
5. Return the removed element.

Python Implementation of Stack

```
class Stack:
    def __init__(self):
        self.top = None

    def push(self, data):
        new_node = Node(data)
        new_node.next = self.top
        self.top = new_node
        print(data, "pushed onto stack")

    def pop(self):
        if self.top is None:
            print("Stack Underflow")
            return None

        data = self.top.data
        self.top = self.top.next
        print(data, "popped from stack")
        return data

    def peek(self):
        if self.top is None:
            print("Stack is empty")
            return None

        return self.top.data

    def display(self):
        current = self.top
        if current is None:
            print("Stack is empty")
            return

        print("Stack (top to bottom):", end=" ")
        while current:
            print(current.data, end=" ")
            current = current.next
        print()
```

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None


# Stack demonstration
stack = Stack()
stack.push(10)
stack.push(20)
stack.push(30)
stack.display()
print("Top element:", stack.peek())
stack.pop()
stack.display()
```

Sample Output

```
10 pushed onto stack
20 pushed onto stack
30 pushed onto stack
Stack (top to bottom): 30 20 10
Top element: 30
30 popped from stack
Stack (top to bottom): 20 10
```

6. Queue Operations Using a Linked List

The main queue operations are:

1. **Enqueue:** Insert an element at the rear.
2. **Dequeue:** Remove an element from the front.
3. **Peek:** View the front element without removing it.
4. **Display:** Print all elements from front to rear.
5. **is_empty:** Check whether the queue contains any elements.

Algorithm for Enqueue

1. Create a new node containing the given value.
2. If the queue is empty, set both front and rear to the new node.
3. Otherwise, link the current rear to the new node.
4. Update rear to point to the new node.

Algorithm for Dequeue

1. Check whether the queue is empty.
2. If empty, report queue underflow.
3. Otherwise, store the front element.
4. Move front to the next node.
5. If front becomes NULL, set rear to NULL as well.
6. Return the removed element.

Python Implementation of Queue

```
class Queue:
    def __init__(self):
        self.front = None
        self.rear = None

    def enqueue(self, data):
        new_node = Node(data)

        if self.rear is None:
            self.front = self.rear = new_node
        else:
            self.rear.next = new_node
```

```
            self.rear = new_node

        print(data, "inserted into queue")

    def dequeue(self):
        if self.front is None:
            print("Queue Underflow")
            return None

        data = self.front.data
        self.front = self.front.next

        if self.front is None:
            self.rear = None

        print(data, "removed from queue")
        return data

    def peek(self):
        if self.front is None:
            print("Queue is empty")
            return None

        return self.front.data

    def display(self):
        current = self.front
        if current is None:
            print("Queue is empty")
            return

        print("Queue (front to rear):", end=" ")
        while current:
            print(current.data, end=" ")
            current = current.next
        print()


# Queue demonstration
queue = Queue()
queue.enqueue(10)
queue.enqueue(20)
queue.enqueue(30)
queue.display()
print("Front element:", queue.peek())
queue.dequeue()
queue.display()
```

Sample Output

```
10 inserted into queue
20 inserted into queue
30 inserted into queue
Queue (front to rear): 10 20 30
Front element: 10
10 removed from queue
Queue (front to rear): 20 30
```

Note: Both programs use the same `Node` class. Run each code example independently, or define `Node` once before both classes in a combined program.

7. Time and Space Complexity

Let (n) represent the number of elements stored in the data structure.

Operation	Stack	Queue
Insertion	Push: O(1)	Enqueue: O(1)
Deletion	Pop: O(1)	Dequeue: O(1)
Peek	O(1)	O(1)
Display	O(n)	O(n)
Space complexity	O(n)	O(n)

Insertion and deletion take constant time because the algorithms update pointers rather than shifting existing elements.

The space complexity is O(n) because each element requires one linked-list node.

8. Advantages

- Dynamic memory allocation.
- No fixed capacity is required in advance.
- Stack push and pop operations take O(1) time.
- Queue enqueue and dequeue operations take O(1) time when both front and rear pointers are maintained.
- No shifting of elements is necessary after deletion.
- Suitable when the number of stored elements changes frequently.

9. Limitations

- Each node requires extra memory for its pointer.
- Nodes may be scattered throughout memory.
- Random access by index is not efficient.
- Pointer management must be implemented correctly.
- Memory allocation can introduce overhead compared with array-based implementations.

10. Real-World Applications

Applications of Stack

- Function calls and recursion.
- Undo and redo operations in text editors.
- Browser history navigation.
- Expression evaluation and syntax parsing.
- Backtracking algorithms.

Applications of Queue

- CPU process scheduling.
- Printer job management.
- Customer service waiting lines.
- Network packet processing.
- Breadth-First Search (BFS) in graphs.

11. Comparison Between Stack and Queue

Feature	Stack	Queue
Principle	LIFO	FIFO
Insertion operation	Push	Enqueue
Deletion operation	Pop	Dequeue
Insertion location	Top	Rear
Deletion location	Top	Front

Main pointers	Top	Front and rear
Example	Stack of plates	People waiting in a line

12. Conclusion

This case study demonstrates the implementation of stack and queue data structures using linked lists. The stack follows the LIFO principle, whereas the queue follows the FIFO principle.

Both data structures support efficient insertion and deletion in O(1) time when implemented with appropriate pointers. Linked lists provide dynamic memory allocation and eliminate the need for a predefined capacity, making them useful for applications in operating systems, scheduling, expression processing, and other areas of computer science.